MODULE 05 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

etcd, high availability and the control plane at work

Quorum arithmetic, HA topologies, and the jobs the control plane does in the background.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 5 of 35

admin@sthithapragna.com

etcd does not need every member to agree. It needs a majority.

admin@sthithapragna.com

What etcd is

- **A distributed key-value store.** Kubernetes keeps every object in it.
- **Raft consensus.** One elected leader; followers replicate its log.
- **A write is committed** only when a majority of members have it.
- **A watch API** streams changes. That is what Kubernetes watches are built on.

> ON THE EXAM
>
> Strong consistency is the point: every API server sees the same committed state.

Quorum arithmetic

MEMBERS	QUORUM NEEDED	FAILURES TOLERATED
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3

ON THE EXAM

Quorum is a majority: half the members, rounded down, plus one. Odd sizes give the best value.

What etcd needs to stay healthy

- **Fast disks.** Every committed write is flushed to disk. Slow disks slow the whole cluster.
- **Low network latency** between members, so they can agree quickly.
- **Regular snapshots.** etcdctl snapshot save, stored off the machine.
- **Tight access.** etcd holds every Secret. Only the API server should reach it.

> ON THE EXAM
>
> If asked which resources matter most for etcd performance: disk I/O and the network.

Stacked versus external etcd

STACKED

etcd on each control plane node

- Fewer machines: at least 3 in total
- Simpler to set up; kubeadm's default
- Lose a node, lose an etcd member too
- Coupled failure

EXTERNAL

etcd on its own hosts

- More machines: at least 3 plus 3, so 6
- More to set up and manage
- Control plane and etcd fail independently
- Decoupled failure

THE TIEBREAKER

Stacked is cheaper. External survives more, at the cost of at least six hosts.

A highly available control plane

01

Load balancer

One address for kubectl and every kubelet.

02

API servers

Three or more, all active at once.

03

Scheduler, controllers

One copy each is leader; the rest wait.

04

etcd

Three or five members, keeping quorum.

ON THE EXAM

Spread control plane instances across failure zones, so one zone going dark does not take the majority with it.

Active-active versus leader election

API SERVER

All active

- Stateless: state lives in etcd
- Any copy can serve any request
- Scale out behind a load balancer
- More copies, more capacity

SCHEDULER, CONTROLLER MANAGER

One leader

- Two leaders would fight over the same Pods
- Leader holds a Lease object
- Standbys take over if it stops renewing
- More copies, faster failover only

THE TIEBREAKER

API servers scale out. The scheduler and controller manager fail over.

How the control plane runs itself: static Pods

- **The puzzle:** the API server runs as a Pod, but Pods need an API server.
- **The answer:** the kubelet reads manifests from a folder on disk and runs them directly.
- With kubeadm, that folder is /etc/kubernetes/manifests on each control plane node.
- A read-only **mirror Pod** appears in kube-system so you can see it with kubectl.

> ON THE EXAM
>
> Static Pods are managed by the kubelet, not the API server. Delete the mirror and it simply comes back.

When a node goes silent

01

Heartbeats

The kubelet renews its Lease in kube-node-lease about every 10 seconds.

02

Silence

Heartbeats stop: power loss, crash, network cut.

03

Unknown

After a grace period of under a minute, Ready becomes Unknown.

04

Tainted

The node controller adds a NoExecute unreachable taint.

05

Evicted

Pods tolerate it for 300 seconds by default, then are evicted.

ON THE EXAM

By default, Pods on a dead node are evicted after about five minutes.

After eviction: who comes back?

OWNED BY A CONTROLLER

Replaced

- Deployment, ReplicaSet, StatefulSet, Job
- Controller sees too few Pods
- Creates a new Pod elsewhere
- New UID, and usually a new name

A BARE POD

Gone

- Created directly, e.g. kubectl run
- No controller watching it
- Deleted with the node
- Nothing recreates it

THE TIEBREAKER

Pods do not heal themselves. Controllers heal them. That is why you almost never create bare Pods.

What garbage collection cleans up

TARGET	HOW
Dependents whose owner is deleted	ownerReferences link them; the collector deletes them
Terminated Pods	Removed once they pass a cluster-wide threshold
Finished Jobs	Removed after ttlSecondsAfterFinished, if set
Unused images and containers	The kubelet cleans them on each node
Released volumes	Deleted when the reclaim policy is Delete
Expired certificate requests	Removed automatically

ON THE EXAM

Owner references are the key idea: delete a Deployment and its ReplicaSets and Pods go too.

Cascading deletion: three choices

KUBECTL DELETE --CASCADE=	WHAT HAPPENS
`background (default)`	Owner deleted at once; dependents cleaned up afterwards.
`foreground`	Dependents deleted first; owner waits, marked as being deleted.
`orphan`	Owner deleted; dependents kept, with their owner reference removed.

> ON THE EXAM
>
> Orphan is how you replace a controller without restarting the Pods it manages.

Finalizers: why deletion sometimes hangs

- **A finalizer** is a key on an object saying: do not remove me until I have cleaned up.
- Delete sets a deletion timestamp; the object stays until every finalizer is removed.
- **Good use:** a volume is not released while a Pod still uses it.
- **Stuck in Terminating** usually means a finalizer whose controller has gone.

> ON THE EXAM
>
> Terminating for hours: look for a finalizer, not a broken scheduler or etcd.

How big can a cluster be?

DIMENSION	DESIGNED LIMIT
Pods per node	110
Nodes	5,000
Total Pods	150,000
Total containers	300,000

ON THE EXAM

Unchanged in the Kubernetes 1.37 docs. Beyond this, run more clusters, not a bigger one.

> Quayside's self-managed cluster runs three etcd members. An engineer proposes adding a fourth member for extra resilience.
>
> The platform lead asks how many member failures the four-member cluster could survive.

How many failures could four members tolerate?

A One, because four members need three for a quorum

B Two, because each extra member adds one failure

C Three, because only one member must remain up

D None, because even counts cannot form a quorum

◆ PAUSE AND DECIDE

How many failures could four members tolerate?

A One, because four members need three for a quorum

Majority of four is three, so only one can fail. Same as three members.

B Two, because each extra member adds one failure

Only every second member adds tolerance. Five would give two.

C Three, because only one member must remain up

etcd needs a majority, not a single survivor.

D None, because even counts cannot form a quorum

Even counts work; they just waste a member.

THE SIGNAL

Going from three to four adds cost and a failure point, not tolerance. Go to five.

> For production, Quayside needs a highly available control plane. Losing an etcd member must never also take out a control plane instance.
>
> They accept running extra machines, but want the minimum number that meets the requirement.

Which layout meets the requirement with the fewest hosts?

A Stacked etcd spread across three control plane nodes

B Stacked etcd spread across four control plane nodes

C External etcd sharing the three control plane hosts

D External etcd: 3 control plane hosts, 3 etcd hosts

◆ PAUSE AND DECIDE

Which layout meets the requirement with the fewest hosts?

A Stacked etcd spread across three control plane nodes

Fewest hosts, but each node failure takes both layers down together.

B Stacked etcd spread across four control plane nodes

Still coupled, and four etcd members add no tolerance.

C External etcd sharing the three control plane hosts

Sharing hosts is just stacked etcd under another name.

D External etcd: 3 control plane hosts, 3 etcd hosts

Decoupled failure, at the minimum of six hosts.

THE SIGNAL

'Must fail independently' means external etcd, and that means at least six hosts.

> At 10:00, a worker node loses power. It was running three web front-end Pods from a Deployment, and a debugging Pod an engineer created with kubectl run.
>
> Default settings apply. The on-call engineer checks the cluster at 10:07.

What will the engineer most likely find?

A. All four Pods have been recreated on other nodes

B. The web Pods were replaced elsewhere; the debug Pod is gone

C. Nothing has changed; Pods wait until the node returns

D. Everything was replaced within ten seconds of power loss

◆ PAUSE AND DECIDE

What will the engineer most likely find?

A. All four Pods have been recreated on other nodes

Nothing owns the debug Pod, so nothing recreates it.

B. The web Pods were replaced elsewhere; the debug Pod is gone

Evicted after about five minutes; only the Deployment recreates its Pods.

C. Nothing has changed; Pods wait until the node returns

True before about five minutes. By 10:07 eviction has happened.

D. Everything was replaced within ten seconds of power loss

There is a grace period, then a 300-second toleration.

THE SIGNAL

Seven minutes is past the five-minute eviction. Ownership decides what comes back.

> Quayside deletes an old test namespace. Hours later, kubectl still shows the namespace as Terminating.
>
> Other namespaces can be created and deleted normally, and all workloads are healthy.

What is the most likely cause?

A. etcd has lost quorum, so the deletion cannot be stored

B. The scheduler is down, so nothing can be removed

C. A finalizer is waiting for a controller that is not acting

D. Foreground deletion always takes several hours

◆ PAUSE AND DECIDE

What is the most likely cause?

A etcd has lost quorum, so the deletion cannot be stored

Then nothing could change anywhere. Other namespaces work fine.

B The scheduler is down, so nothing can be removed

The scheduler places Pods. It plays no part in deletion.

C A finalizer is waiting for a controller that is not acting

Deletion was accepted but is blocked until the finalizer is removed.

D Foreground deletion always takes several hours

It waits for dependents, which normally takes seconds.

THE SIGNAL

One object stuck in Terminating while everything else works: a finalizer.

Module 5 in five lines

- etcd needs a **majority**: 3 members tolerate 1 failure, 5 tolerate 2. Never 4.
- **Stacked**: 3 hosts, coupled. **External**: 6 hosts, independent.
- **API servers are all active**; scheduler and controllers use leader election.
- Silent node: Pods **evicted after about 5 minutes**; only owned Pods come back.
- **Owner references** drive garbage collection; **finalizers** can block deletion.

We have the whole cluster. Next: the Pod, the smallest thing it runs.

admin@sthithapragna.com